



**Escola de Engenharia**  
**Programa de Pós-Graduação em Engenharia Elétrica**

EEE889 - Eletromagnetismo Computacional

Raphael Henrique Braga Leivas

## **LISTA 5**

Belo Horizonte  
27 de maio de 2026

## SUMÁRIO

|                                               |          |
|-----------------------------------------------|----------|
| <b>1 EXERCÍCIO 9.2 . . . . .</b>              | <b>3</b> |
| <b>ANEXO A CÓDIGO EXERCÍCIO 9.2 . . . . .</b> | <b>6</b> |

## 1 EXERCÍCIO 9.2

Todo o código usado nesse exercício encontra-se no Anexo A.

Considerando o problema do exercício 8.2 em que temos um domínio 2D quadrado e queremos simular o modo  $TM_z$  com o FDTD, usamos os seguintes parâmetros de discretização:

- $N_x = 200$  ,  $N_y = 200$
- $\Delta y = \Delta x = 0.1 \cdot \lambda = 0.03 \text{ m}$
- $\Delta t = \frac{0.99}{c\sqrt{(\Delta x)^2 + (\Delta y)^2}} = 70 \text{ ps}$  (condição CFL 2D)
- $N_t = 250$

A fonte aplicada no centro do domínio é uma função gaussiana dada por

$$f(t) = e^{-\frac{(t-1.8 \text{ ns})^2}{(0.6 \text{ ns})^2}}$$

que tem um pico em  $t = 1.8 \text{ ns}$  e uma largura de aproximadamente  $0.6 \text{ ns}$ .

Os parâmetros da UPML adotados foram os seguintes:

- $N_{pml} = 60$  ,  $m = 3$
- $\sigma_{max} = 0.054$
- $\sigma(x) = \sigma_{max} \left( \frac{x}{N_{pml}} \right)^m$

A propagação obtida está exibida na Figura 1, onde podemos observar que o campo se propaga para fora do domínio sem reflexões significativas, indicando que os PMLs estão funcionando corretamente. Com uma ponta de prova próximo ao centro do grid, validamos que a reflexão é praticamente nula, dado que o campo fica em zero após a passagem do pulso gaussiano, sem oscilações residuais, como mostra a Figura 2.

A Figura 3 mostra a evolução do campo  $E_z$  ao longo do tempo, em um corte horizontal no eixo  $x$  passando pelo centro da cavidade. As linhas verticais vermelhas indicam a região onde o campo é absorvido pelos PMLs, e podemos observar que o campo se propaga para fora do domínio sem reflexões significativas, indicando que os PMLs estão funcionando corretamente.

Figura 1 – Propagação de  $E_z$  ao longo do grid, sem reflexão nas bordas.

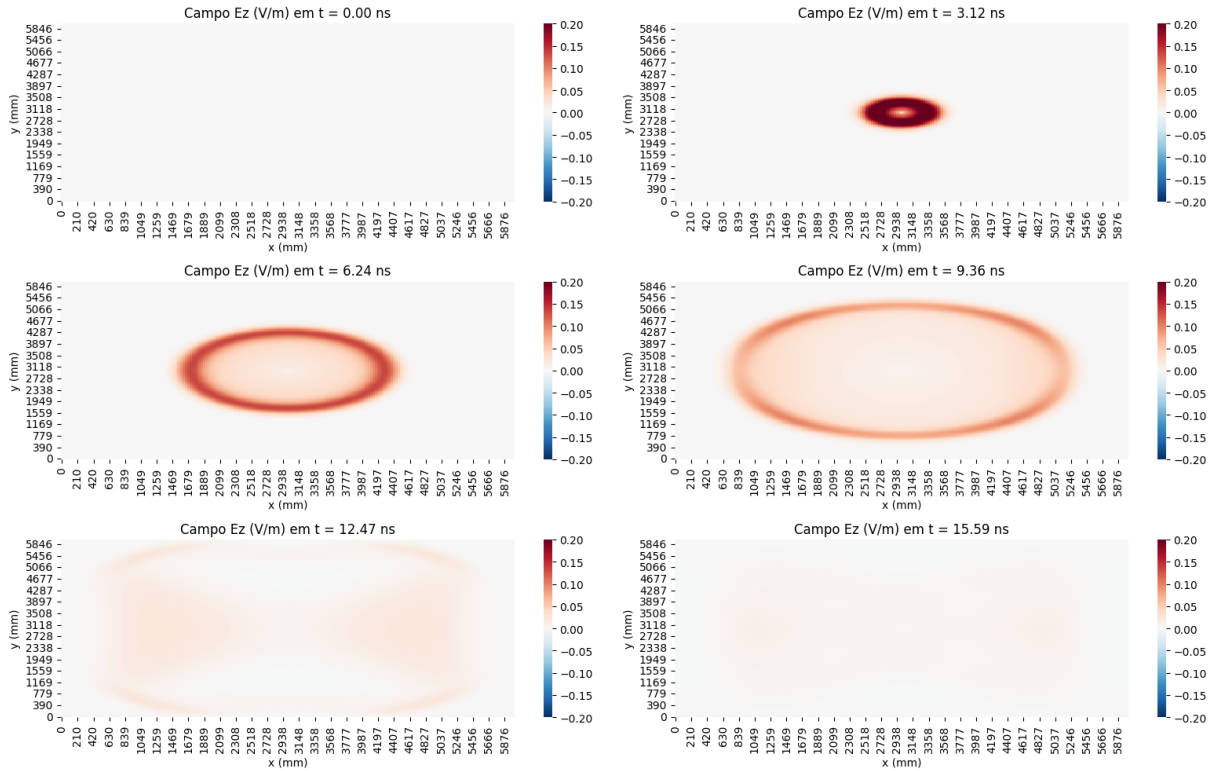


Figura 2 – Amplitude de  $E_z$  no centro do grid em função do tempo.

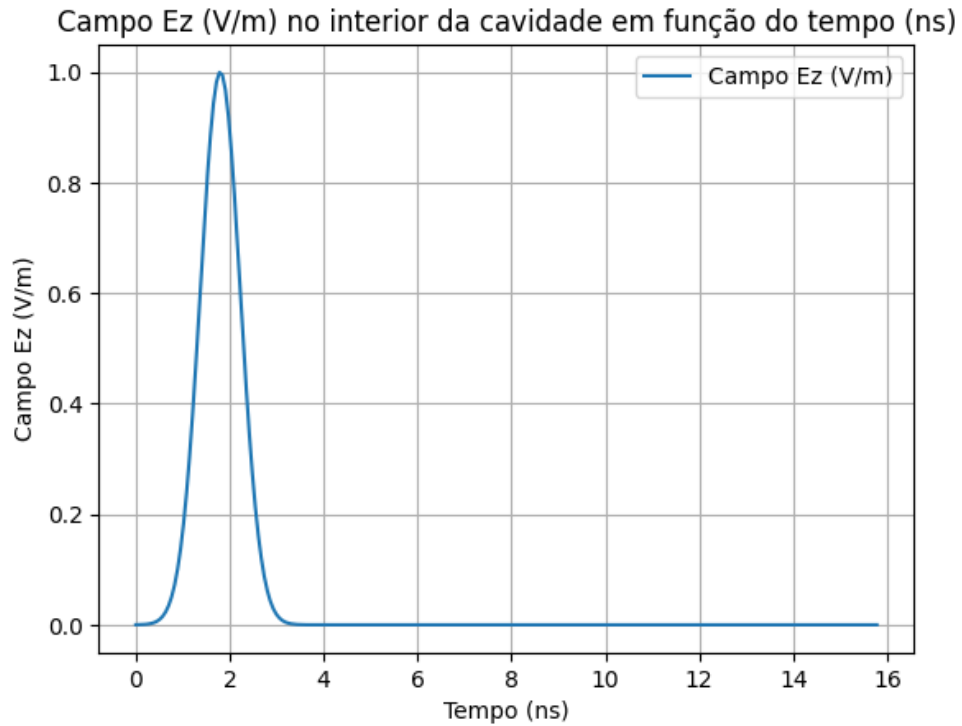
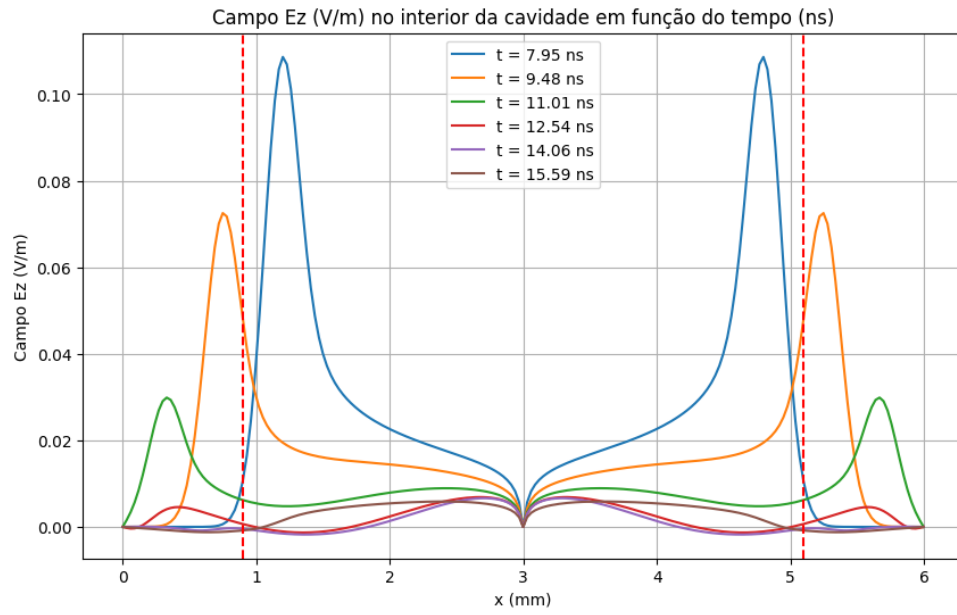


Figura 3 – Decaimento do campo  $E_z$  ao longo do eixo  $x$  ao entrar nas regiões de PML.



## ANEXO A – CÓDIGO EXERCÍCIO 9.2

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.fft import fft, fftfreq

# --- parâmetros físicos ---
eps = 8.854187817e-12 # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7 # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)

# --- domínio ---
min_lambda = vp / 1e9 # comprimento de onda mínimo para 1 GHz = 3e-1 m
Nx = Ny = 200
dx = dy = 0.1 * min_lambda
dt = 0.9 / (vp * np.sqrt(1/dx**2 + 1/dy**2))
Nt = 250
L = Nx * dx
T = Nt * dt

# valores do tempo para salvar os snapshots e montar os heatmaps
n_snapshots = np.linspace(Nt // 2, Nt - 5, 6, dtype=int)
Ez_snapshots = []

cx, cy = Nx // 2, Ny // 2
px, py = cx, cy # pontas de prova

grid_x_Ez = np.linspace(0, L, Nx + 1)
grid_y_Ez = np.linspace(0, L, Ny + 1)
grid_t_Ez = np.linspace(0, T, Nt + 1)

grid_x_Hx = np.linspace(0, L, Nx + 1)
grid_y_Hx = np.linspace(dy/2, L - dy/2, Ny)
grid_t_Hx = np.linspace(dt/2, T - dt/2, Nt)

grid_x_Hy = np.linspace(dx/2, L - dx/2, Nx)
grid_y_Hy = np.linspace(0, L, Ny + 1)
grid_t_Hy = np.linspace(dt/2, T - dt/2, Nt)

# --- campos ---
Ez_curr = Ez_next = np.zeros((grid_x_Ez.shape[0], grid_y_Ez.shape[0]))
Hx_curr = Hx_next = np.zeros((grid_x_Hx.shape[0], grid_y_Hx.shape[0]))
Hy_curr = Hy_next = np.zeros((grid_x_Hy.shape[0], grid_y_Hy.shape[0]))

# ---- parâmetros da PML ----
Npml = 60
m = 3 # ordem polinomial
R0 = 1e-8 # reflexão alvo
sigma_max_x = -1 * (m + 1) * np.log(R0) / (2 * np.sqrt(mu / eps) * Npml * dx)
sigma_max_y = -1 * (m + 1) * np.log(R0) / (2 * np.sqrt(mu / eps) * Npml * dy)

# ---- vetores para salvar os sigmas ----
# sigma = 0 dentro da regioao, diferente de zero na PML
sigma_x = np.zeros(Nx + 1)
sigma_y = np.zeros(Ny)

for i in range(Npml):
    x = (Npml - i) / Npml
    sigma_x[i] = sigma_max_x * x**m

```

```

sigma_x[Nx - i] = sigma_max_x * x**m

for j in range(Npml):
    y = (Npml - j) / Npml
    sigma_y[j] = sigma_max_y * y**m
    sigma_y[Ny - j - 1] = sigma_max_y * y**m

# buffer de integração
l_curr = l_next = np.zeros((grid_x_Hx.shape[0], grid_y_Hx.shape[0]))

# ponta de prova para coletar pontos para a FFT no centro da cavidade
probe = []

# marcha no tempo
for n in range(Nt-1):
    # calcula os coeficientes
    F1_x = ((2 * eps - dt * sigma_x[:]) / (2 * eps + dt * sigma_x[:]))
    F2_x = (1 / mu) * ( (2 * eps * dt) / (2 * eps + dt * sigma_x[:]) )

    F1_y = ((2 * eps - dt * sigma_y[:]) / (2 * eps + dt * sigma_y[:]))
    F2_y = (1 / mu) * ( (2 * eps * dt) / (2 * eps + dt * sigma_y[:]) )

    sigma_y_appended = np.append(sigma_y, [sigma_y[-1]])
    sigma_e = (sigma_x[1:-1][:, None] + sigma_y_appended[1:-1][None, :]) / 2
    G1 = (2 * eps - dt * sigma_e) / (2 * eps + dt * sigma_e)
    G2 = 2 * dt / (2 * eps + dt * sigma_e)

    Hy_next[:, :] = F1_x * Hy_curr[:, :] + F2_x * (
        (Ez_curr[1:, :] - Ez_curr[:, -1]) / dx
    )

    Hx_next[:, :] = F1_y * Hx_curr[:, :] - F2_y * (
        Ez_curr[:, 1:] - Ez_curr[:, -1]
    ) / dy + (sigma_y[:] * (dt ** 2) / (mu * eps)) * l_curr[:, :]

    # Hx_next[:, :] = Hx_curr[:, :] - (dt/(mu*dy)) * (
    #     Ez_curr[:, 1:] - Ez_curr[:, -1]
    # ) + (sigma_x[1:] * (dt ** 2) / (mu * eps)) * l_curr[:, :]

    Ez_next[1:-1, 1:-1] = G1 * Ez_curr[1:-1, 1:-1] + G2 * (
        (Hy_next[1:, 1:-1] - Hy_next[:, -1, 1:-1]) / dx
        - (Hx_next[1:-1, 1:] - Hx_next[1:-1, :, -1]) / dy
    )

    # Ez_next[1:-1, 1:-1] = G1 * Ez_curr[1:-1, 1:-1] + G2 * (
    #     (Hy_next[1:, 1:-1] - Hy_next[:, -1, 1:-1]) / dx -
    #     (Hx_next[1:-1, 1:] - Hx_next[1:-1, :, -1]) / dy
    # )

    l_next[:, :] = l_curr[:, :] + (Ez_curr[:, 1:] - Ez_curr[:, -1])

# --- fontes ---
frequency = 1e9
source_gaussiano = np.exp(-((n * dt - 1.8e-9)**2) / (0.6e-9)**2)
source_senoidal = np.sin(2 * np.pi * frequency * n * dt)
Ez_next[cx, cy] = source_gaussiano

# PEC nas bordas externas do grid
Ez_next[0, :] = Ez_next[Nx, :] = Ez_next[:, 0] = Ez_next[:, Ny] = 0

```

```
# salva o valor no centro para a FFT
probe.append(Ez_next[px, py])

# atualiza os campos para o próximo passo
Ez_curr, Ez_next = Ez_next, Ez_curr
Hx_curr, Hx_next = Hx_next, Hx_curr
Hy_curr, Hy_next = Hy_next, Hy_curr
I_curr, I_next = I_next, I_curr

# salva para os snapshots
if n in n_snapshots:
    Ez_snapshots.append(Ez_curr.copy())
```